QA Finance — System Architecture

A QA automation platform for financial applications where customers define their testing requirements, an LLM generates production-ready Playwright test scripts, and execution occurs within an isolated Docker sandbox while providing a live browser stream.

---

1. High-Level Architecture

The QA Finance platform follows a modular, service-oriented architecture consisting of four primary layers:

- **Frontend Application** — React, TypeScript, and Vite
- **Backend API** — Express and TypeScript
- **AI Services** — Claude, Gemini, and Groq
- **Execution Layer** — Docker-based Playwright runtime

The frontend communicates with the backend through secure REST APIs for standard application operations and WebSocket connections for real-time execution updates. Authentication is implemented using short-lived JWT access tokens and rotating HTTP-only refresh cookies.

The backend orchestrates authentication, AI-assisted test generation, Docker execution, database persistence, and real-time event streaming.

Generated Playwright scripts are executed inside isolated Docker containers, ensuring secure, reproducible, and disposable execution environments. Execution progress, browser screenshots, AI interactions, and test status updates are streamed back to the client through WebSockets.

PostgreSQL serves as the primary persistence layer for user accounts, authentication tokens, automation runs, generated scripts, execution history, and test results. Email verification and password recovery workflows are handled through Resend.

---

2. Repository Structure

The solution is organized into two independent Node.js applications.

```
QA Project/
    qa-finance/
    server/
```

Frontend (`qa-finance/`)

The frontend is implemented using React 19, TypeScript, Vite, and Tailwind CSS v4. It provides the complete user interface, including authentication, dashboard management, automation creation, test execution monitoring, reporting, and application settings.

Backend (`server/`)

The backend is implemented using Express and TypeScript and is organized into modular components.

Key modules include:

- Authentication
- Automation orchestration
- AI provider integrations
- WebSocket services
- Shared libraries
- Middleware
- Prisma schema
- Docker execution environment

Both projects maintain independent package configurations and are developed independently using their respective development servers.

---

3. Authentication Architecture

The authentication subsystem supports multiple authentication mechanisms while maintaining a unified user model.

Supported authentication methods include:

- Email and password authentication
- Google OAuth
- GitHub OAuth

Passwords are securely hashed using bcrypt before storage. User authentication relies on short-lived JWT access tokens together with rotating refresh tokens stored as hashed values within the database and delivered through HTTP-only cookies.

Email verification and password reset functionality are implemented using one-time cryptographically secure tokens delivered via Resend with configurable expiration periods.

OAuth authentication follows the Authorization Code Flow without external authentication frameworks. CSRF protection is implemented using state validation, while provider accounts are linked using verified email addresses to ensure a single identity regardless of authentication method.

For security, frontend access tokens remain in memory only. Following a browser refresh, the application silently restores the authenticated session by redeeming the refresh cookie through the authentication endpoint.

The authentication module maintains dedicated tables for users, refresh tokens, password reset tokens, email verification tokens, and OAuth account mappings.

---

4. Automation Engine

The automation engine represents the platform's core capability. It transforms user requirements into executable Playwright test suites through an AI-assisted workflow.

Each automation session is implemented as a structured conversation with an LLM. The model operates using two predefined tools:

- **ask_question()** — Requests additional information from the customer whenever clarification is required.
- **submit_test_code()** — Returns the completed Playwright test script together with a human-readable summary.

This abstraction is implemented consistently across all supported AI providers, allowing each provider to expose an identical interface while remaining interchangeable.

At present, a single provider is selected as the active implementation during deployment. Provider switching is performed through a single service import without affecting the remainder of the automation pipeline.